

Arquitectura SAVCNG 2.0

1. Propósito de la arquitectura

La arquitectura SAVCNG 2.0 organiza el proyecto ExcelDNA como un conjunto de capas con responsabilidades separadas: una vista de Windows Forms orientada a orquestación, una capa central de utilidades para operaciones repetitivas sobre Excel, un conjunto de estrategias de validación intercambiables y un DTO de salida para transportar el resultado de cada operación.

El objetivo principal es mantener el código de interfaz liviano, aislar la lógica de negocio de Excel Interop y aplicar una disciplina estricta de memoria COM para evitar referencias huérfanas, bloqueos de Excel y fugas de memoria.

2. Vista general de capas

El proyecto puede leerse en cuatro bloques funcionales:

- `FrmValidaciones.cs`: punto de entrada de la experiencia de usuario y orquestador de acciones.
- `Core/ExcelHelper.cs`: fachada técnica para servicios comunes de Interop, como traducción de fórmulas, localización de preguntas y liberación COM.
- `Validaciones/`: conjunto de estrategias concretas que implementan una misma interfaz contractual.
- `Core/ResultadoValidacion.cs`: objeto de transferencia de datos que comunica el resultado de ejecución hacia la vista.

`AuditoriaCenso.cs` complementa estas capas como motor de bitácora y persistencia de trazabilidad dentro del propio libro de Excel.

3. Patrón MVP en FrmValidaciones

`FrmValidaciones` actúa como una vista "Dumb View" dentro de una variante práctica de MVP. El formulario contiene la interacción visual, el estado inmediato de la interfaz y la captura de eventos, pero no concentra la lógica de validación ni la manipulación compleja del libro.

3.1. Qué hace la vista

El formulario recibe el libro abierto por constructor y lo guarda en `_libroCenso`. A partir de ahí, su trabajo consiste en:

- mostrar estado al usuario, como el nombre del censo cargado;
- capturar selección de rangos desde Excel;
- permitir elegir una validación concreta;
- ejecutar el servicio o estrategia correspondiente;
- dibujar el resultado recibido en un `MessageBox` o en controles de pantalla.

3.2. Evidencia de Dumb View

La vista no calcula reglas de validación, no construye fórmulas complejas y no decide cómo aplicar formatos o validaciones internas. En su lugar:

- delega la resolución de preguntas al helper central;
- selecciona una estrategia según el checkbox activo;
- consume un `ResultadoValidacion` devuelto por la capa de lógica;
- actualiza etiquetas, grillas y avisos según el resultado.

Esto se observa especialmente en `btnAplicar_Click`, donde el formulario solo actúa como enrutador de estrategia, y en `CargarEstadoDelCenso`, donde únicamente enlaza una tabla a la grilla de auditoría.

3.3. Rol de orquestación

El formulario concentra decisiones de UI, no de negocio. Por ejemplo:

- valida que exista un libro cargado;
- verifica que el usuario haya capturado un rango;
- habilita o deshabilita estados visuales;
- refresca el `DataGridView`;
- mantiene el checkbox exclusivo seleccionado.

Ese comportamiento es consistente con MVP: la vista sigue siendo delgada, y la lógica reusable vive fuera de ella.

4. Patrón Strategy en Validaciones

La carpeta `Validaciones` implementa el patrón Strategy mediante la interfaz `IValidacionExcel`.

4.1. Contrato común

`IValidacionExcel` define un único método:

- recibe `Excel.Application`, `Excel.Workbook` y `Excel.Range`;
- devuelve un `ResultadoValidacion`;
- permite que cada validación resuelva su propia lógica sin cambiar la vista.

Este contrato hace que el botón “Aplicar” no necesite conocer detalles internos de cada validación, solo necesita elegir una clase concreta que cumpla la interfaz.

4.2. Implementaciones concretas

Las clases `ValidacionDecimales`, `ValidacionCatalogos`, `ValidacionNS`, `ValidacionFormatoTexto`, `ValidacionSumas`, `ValidacionFechas`, `ValidacionBloqueos`, `ValidacionBlancos`, `ValidacionEspecifique` y `ValidacionCoordenadasGeograficas` representan estrategias independientes.

Cada una encapsula un tipo de regla distinta, por ejemplo:

- validación numérica;
- listas de catálogo;
- marcación de NS;
- validación de texto alfanumérico;
- sumatorias cruzadas;
- restricciones geográficas;
- control de blancos y bloqueos.

4.3. Selección dinámica de estrategia

En `FrmValidaciones.cs`, el método `btnAplicar_Click` selecciona la estrategia mediante una cadena de `if / else if` sobre los checkboxes activos. Esa selección ocurre en tiempo de ejecución, lo que permite:

- agregar nuevas validaciones sin alterar el contrato del formulario;
- intercambiar comportamientos sin tocar la vista;
- mantener aislada la implementación de cada regla.

4.4. Beneficio arquitectónico

El patrón Strategy evita un formulario monolítico lleno de lógica específica. Cada clase de validación puede evolucionar, corregirse o auditarse de forma independiente, mientras el formulario conserva un punto único de invocación.

5. Patrón Façade en ExcelHelper

`Core/ExcelHelper.cs` funciona como una fachada técnica de Excel Interop.

5.1. Responsabilidad principal

La clase centraliza operaciones repetidas o delicadas sobre objetos COM de Excel, de modo que el resto del sistema no tenga que repetir lógica de bajo nivel ni manipular detalles innecesarios de Interop.

5.2. Servicios expuestos

Las responsabilidades observadas en el código son:

- `TraducirFormulaLocal`: traduce una fórmula universal en inglés a su representación local en Excel.
- `ObtenerNumeroPregunta`: identifica el número o identificador de pregunta desde el rango seleccionado.
- `LiberarCom`: destruye explícitamente referencias COM de forma segura.

5.3. Papel como fachada

La fachada simplifica el consumo de Excel en toda la solución:

- las validaciones no deben conocer cómo traducir una fórmula por idioma local;
- el formulario no debe saber cómo recorrer filas para detectar la pregunta;
- cualquier capa puede liberar referencias COM mediante un único método estándar.

Esto reduce duplicación, unifica la política de memoria y hace que las validaciones se lean como reglas de negocio y no como scripts de Interop dispersos.

5.4. Centralización de la política Zero Leaks

La propia fachada también funciona como punto normativo para la memoria COM. `LiberarCom` se usa de forma reiterada en el proyecto para liberar hojas, rangos, celdas y condiciones de formato.

6. DTO de transferencia: ResultadoValidacion

`Core/ResultadoValidacion.cs` define el estándar de respuesta entre la lógica de validación y la interfaz.

6.1. Objetivo del DTO

El DTO encapsula el resultado de una ejecución sin acoplar la lógica al formulario. En vez de devolver mensajes por medio de la UI, la estrategia retorna un objeto con información estructurada.

6.2. Campos del DTO

El objeto contiene tres datos:

- `Exito`: indica si la operación terminó correctamente.
- `Mensaje`: texto que la vista puede mostrar al usuario.
- `AlertaInyectada`: marca si la validación dejó una alerta visual o condición auxiliar aplicada.

6.3. Uso arquitectónico

La vista consume el DTO para decidir:

- qué icono mostrar;
- qué título usar;
- si debe notificar éxito, advertencia o error;
- si debe limpiar o desmarcar controles.

Las estrategias, por su parte, devuelven un DTO incluso en casos de cancelación o error, evitando que el control de flujo dependa de múltiples `MessageBox` dispersos en la lógica interna.

7. AuditoriaCenso como motor de trazabilidad

`AuditoriaCenso.cs` registra y recupera el historial del censo en una hoja oculta del mismo libro.

7.1. Registro

`RegistrarAccion` inserta un evento de auditoría con fecha, pregunta, tipo de validación, rango afectado, funcionalidad Excel y usuario de red.

7.2. Lectura

`ObtenerHistorialCenso` carga la bitácora en memoria como `DataTable` para ser consumida por la grilla de la interfaz.

7.3. Papel dentro de la arquitectura

Aunque no es una fachada ni una estrategia, sí es una pieza de infraestructura de la solución:

- preserva trazabilidad;
- soporta lectura y escritura dentro del libro;
- opera con disciplina COM;
- alimenta la vista con datos ya estructurados.

8. Regla Zero Leaks para COM Interop

La arquitectura SAVCNG 2.0 impone una política estricta de memoria para Excel Interop.

8.1. Principio general

Todo objeto COM obtenido desde Excel debe liberarse explícitamente. Esto aplica a:

- `Workbook`;
- `Worksheet`;
- `Range`;
- `FormatCondition`;
- colecciones intermedias;
- objetos auxiliares usados para traducción, búsqueda o escritura.

8.2. Liberación obligatoria en finally

La regla operativa observada en el código es:

- adquirir el objeto COM dentro de un bloque `try`;
- liberar el objeto en `finally`;
- tolerar fallos de liberación sin romper la experiencia del usuario;
- evitar dejar referencias activas que sobrevivan al método.

`ExcelHelper.LiberarCom` es el mecanismo estandarizado para cumplir esta norma.

8.3. Prohibición de foreach sobre colecciones COM

La arquitectura prohíbe usar `foreach` sobre colecciones COM cuando eso pueda dejar enumeradores vivos o referencias implícitas sin liberar. En su lugar, se exige:

- recorrer por índice con `for`;
- capturar cada objeto COM en una variable explícita;
- liberar manualmente cada referencia en `finally`.

Este criterio aparece reflejado en varias validaciones, donde se sustituyen recorridos enumerables por ciclos indexados para proteger la memoria.

8.4. Criterio de aplicación

La política Zero Leaks busca evitar:

- Excel colgado al cerrar el libro;
- procesos fantasma de Excel;
- acumulación de referencias RCW;
- degradación de rendimiento en operaciones repetidas.

9. Flujo de ejecución típico

Un flujo normal de la arquitectura es el siguiente:

1. El usuario abre el formulario principal.
2. `FrmValidaciones` recibe el libro activo y carga la bitácora.

3. El usuario captura un rango y selecciona una validación.
4. La vista resuelve qué estrategia concreta ejecutar.
5. La estrategia aplica reglas en Excel, usa `ExcelHelper` cuando necesita traducción o limpieza, y registra la operación en `AuditoriaCenso`.
6. La estrategia devuelve un `ResultadoValidacion`.
7. La vista presenta el resultado al usuario y actualiza su estado visual.

10. Conclusión técnica

La Arquitectura SAVCNG 2.0 combina MVP, Strategy, Façade y DTO para mantener el proyecto de ExcelDNA controlado y predecible.

- MVP mantiene la interfaz liviana.
- Strategy encapsula cada tipo de validación.
- Façade concentra operaciones repetidas y políticas de Interop.
- DTO formaliza la salida de cada validación.
- Zero Leaks disciplina la liberación de objetos COM.

El resultado es una base más mantenible para crecer con nuevas reglas de validación sin volver más compleja la capa de presentación ni comprometer la estabilidad de Excel.